UNIVERSIDAD DON BOSCO

ESCUELA DE COMPUTACIÓN

Programación Orientada a Objetos POO104 G02T



APLICACIÓN DE REGISTROS MÉDICOS
PROYECTO DE CÁTEDRA – PRIMERA FASE

DOCENTE:

Ing. Vladimir Díaz Campo

INTEGRANTES:

N°	Nombre Completo	Carnet
1	Escobar Ortega, Emmanuel Alexander	EO260404
2	Cosme Palacios, Pedro Aristides	CP240499
3	Quintanilla Avalos, Fernando Josue	QA221370
4	Pacas Ramos, Aaron Jaziel	PR252213
5	Guerrero Polanco, José Manuel	GP170487

Viernes 28 de agosto de 2026

2. Descripción del proyecto.

2.1 Contexto y Problemática.

Dentro del área de salud, el registro y seguimiento de la información médica de los pacientes es un proceso crítico e importante que influye directamente en la calidad de la atención. Actualmente, muchos centros médicos de atención primaria o clínicas independientes continúan utilizando expedientes físicos en papel o herramientas de hoja de cálculo no digitales.

Esta práctica genera diversas problemáticas operativas:

- **Pérdida y desorganización de datos:** Los documentos físicos están expuestos a extravíos, deterioro o duplicidad de expedientes.
- **Ineficiencia en el tiempo de consulta:** La búsqueda manual de la ficha clínica de un paciente retrasa la atención y complica la revisión rápida de antecedentes, alergias, tratamientos previos y recetas.
- **Errores de captura e inconsistencia:** La falta de validaciones en la entrada de datos (como letras en campos numéricos, formatos de fecha incorrectos o valores fuera de rangos médicos lógicos) provoca incoherencias en la información registrada.
- **Dificultad en el seguimiento de recetas y diagnósticos:** No contar con un enlace claro entre el médico tratante, la consulta realizada y los medicamentos prescritos limita el control adecuado del historial del paciente.

La creación de este Sistema de Gestión de Historiales Clínicos utilizando POO surge como una solución que centraliza, válida y organiza de forma eficiente la información clínica. Permite registrar pacientes, médicos, consultas y recetas dentro de una estructura confiable y fácil de administrar.

2.2 Objetivos del Sistema.

Objetivo General:

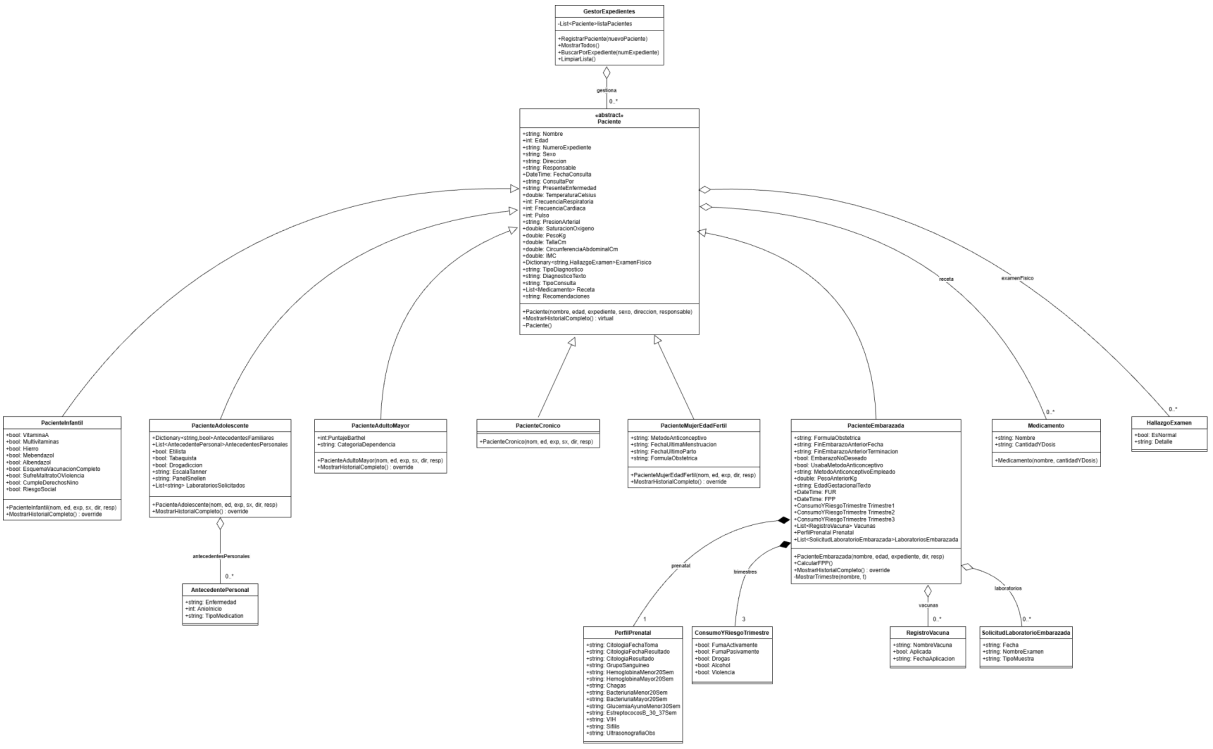
- Desarrollar e implementar un sistema de escritorio en C# para la gestión de historiales clínicos, aplicando los principios de la Programación Orientada a Objetos (POO), que permita administrar de forma correcta, organizada y segura la información de pacientes, médicos, consultas y recetas médicas.

Objetivos Específicos:

1. Estructurar un modelo de datos orientado a objetos(POO) aplicando conceptos de abstracción, herencia, encapsulamiento y agregación para representar adecuadamente la lógica de negocio y las entidades del sistema.
2. Diseñar una interfaz gráfica de usuario en consola fácil de usar e intuitiva, que facilite el registro, la consulta y la actualización de los historiales clínicos.
3. Garantizar la integridad de los datos mediante el manejo de excepciones implementando bloques de control try-catch-finally en los flujos críticos de captura de datos para prevenir caídas del sistema ante errores de formato (FormatException) o valores fuera de rango.

3. Modelado Orientado a Objetos.

3.1 Diagrama de Clases UML



3.2 Breve descripción de las clases principales y sus relaciones (Herencia, Asociaciones, Agregaciones).

El sistema de gestión de historiales clínicos se desarrolla como entidad principal a la clase base **Paciente**, a la cual se derivan las especializaciones de los distintos tipos de atención médica de la clínica. A continuación, se presenta la descripción de las clases principales y sus subclases:

- **Paciente (Clase Principal / Base Abstracta):** Representa la entidad central y abstracta dentro del sistema. Define los atributos comunes y compartidos por todos los expedientes, tales como **(Nombre, Edad, NumeroExpediente, Sexo, Direccion, Responsable, FechaConsulta, ConsultaPor,**

PresenteEnfermedad) y los registros de signos vitales **(TemperaturaCelsius, FrecuenciaRespiratoria, FrecuenciaCardiaca, Pulso, PresionArterial, SaturacionOxigeno, PesoKg, TallaCm, CircunferenciaAbdominalCm).**

3.2.1 Subclases (Heredadas por el Paciente).

1. **PacienteEmbarazada:** Describe la atención obstétrica y prenatal, además incorpora atributos específicos como, **FormulaObstetrica, FinEmbarazoAnteriorFecha, FinEmbarazoAnteriorTerminacion, EmbarazoNoDeseado, UsabaMetodoAnticonceptivo, MetodoAnticonceptivoEmpleado, PesoAnteriorKg, EdadGestacionalTexto, FUR, FPP, Trimestre1, Trimestre2, Trimestre3, Vacunas, Prenatal y LaboratoriosEmbarazada.**

2. **PacienteInfantil:** Representa a los pacientes en etapa pediátrica, incorporando los datos específicos de control, dentro del desarrollo infantil y vacunación.

3. **PacienteAdolescente:** Estructura a los pacientes y adolescentes, incluyendo atributos para la evaluación de antecedentes como, **AntecedentesFamiliares (Dictionary<string, bool>) y AntecedentesPersonales (List<AntecedentePersonal>).**

4. **PacienteAdultoMayor:** Implementada para la atención de adultos mayores, incluyendo los atributos para la evaluación gerontológica y el seguimiento de la salud integral.

5. **PacienteCronico:** Particularización para pacientes con enfermedades de larga duración, registrando datos sobre su patología principal y revisiones periódicas.

6. **PacienteMujerEdadFertil:** Registra la atención en mujeres de edad fértil, sin embargo, se adiciona atributos específicos como, **MetodoAnticonceptivo, FechaUltimaMenstruacion, FechaUltimoParto y FormulaObstetrica.**

3.2.2 Clases de Gestión y Soporte.

1. **GestorExpediente (Clase de Gestión):** Responsable de administrar el registro general de expedientes dentro del sistema, mediante una lista de tipo **List<Paciente>**, lo que permite insertar, buscar, listar y limpiar expedientes haciendo uso del polimorfismo.

2. **CapturadorDatos (Clase Auxiliar / Soporte):** Almacena los métodos para la lectura y la validación de datos ingresados desde la consola (**LeerEntero**, etc.), manejando las excepciones del sistema.

3.2.3 Justificación de las Relaciones Orientadas a Objetos.

- **Herencia.**

    1. **Paciente — PacienteEmbarazada, PacienteInfantil, PacienteAdolescente, PacienteAdultoMayor, PacienteCronico, PacienteMujerEdadFertil:** Se utiliza la herencia simple, donde el **Paciente** funciona como una clase abstracta. Las 6 subclases heredan sus atributos generales y los relacionados con los signos vitales. Este mecanismo previene la duplicación de código y permite aplicar el polimorfismo, haciendo posible recopilar diferentes tipos de pacientes, por medio de una lista **List<Paciente>** y ejecutar el método **MostrarHistorialCompleto()** según la instancia correcta.

- **Agregación y Composición.**

    1. **GestorExpedientes — Paciente (Agregación):** La clase **GestorExpedientes** conserva una lista **List<Paciente>** para gestionar las distintas instancias de los pacientes. Se clasifica como una agregación ya que las instancias del **Paciente** representan los expedientes médicos mediante un ciclo de vida propio, dentro de la lógica del sistema.

    2. **Paciente (Clases Auxiliares / Composición):** Integran tipos especializados (**ConsumoRiesgoTrimestre, PerfilPrenatal o AntecedentePersona**l) para estructurar la información detallada dentro de la misma entidad principal.

- **Asociación.**

    1. **GestorExpedientes — CapturadorDatos:** Existe una relación funcional entre la clase gestora y la clase de captura, ya que permite validar y procesar la entrada de datos por consola.

4. Evidencia de implementación técnica.

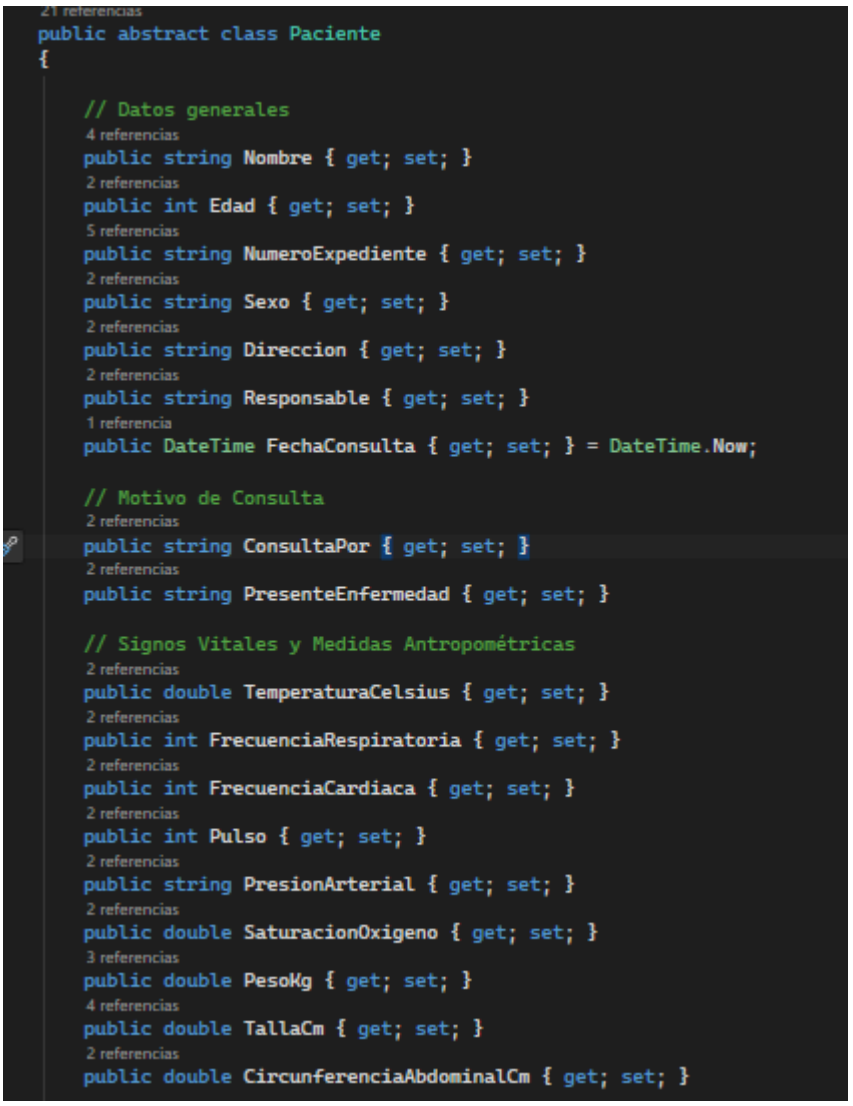
4.1. Clases, Objetos y sobrecarga.

Jerarquía de clases y objetos.

En nuestra solución se utilizaron herencias y encapsulamiento para modelar distintos perfiles de pacientes para la unidad de salud correspondiente. Se definió una clase base abstracta llamada “Paciente”, de la cual derivan seis subclases especializadas (PacienteEmbarazada, PacienteInfantil, PacienteAdolescente, PacienteAdultoMayor, PacienteCronico y PacienteMujerEdadFertil).

- Clase Base y atributos de apoyo (Paciente.cs)

  La clase abstracta “Paciente” no puede ser instancia directamente, sino que provee los atributos compartidos (nombre, edad, expediente, dirección, etc.) e integra relaciones de composición con clases auxiliares para administrar “SignosVitales”, “ExamenFisicoItem”, “Datos generales”.



```
21 referencias
public abstract class Paciente
{

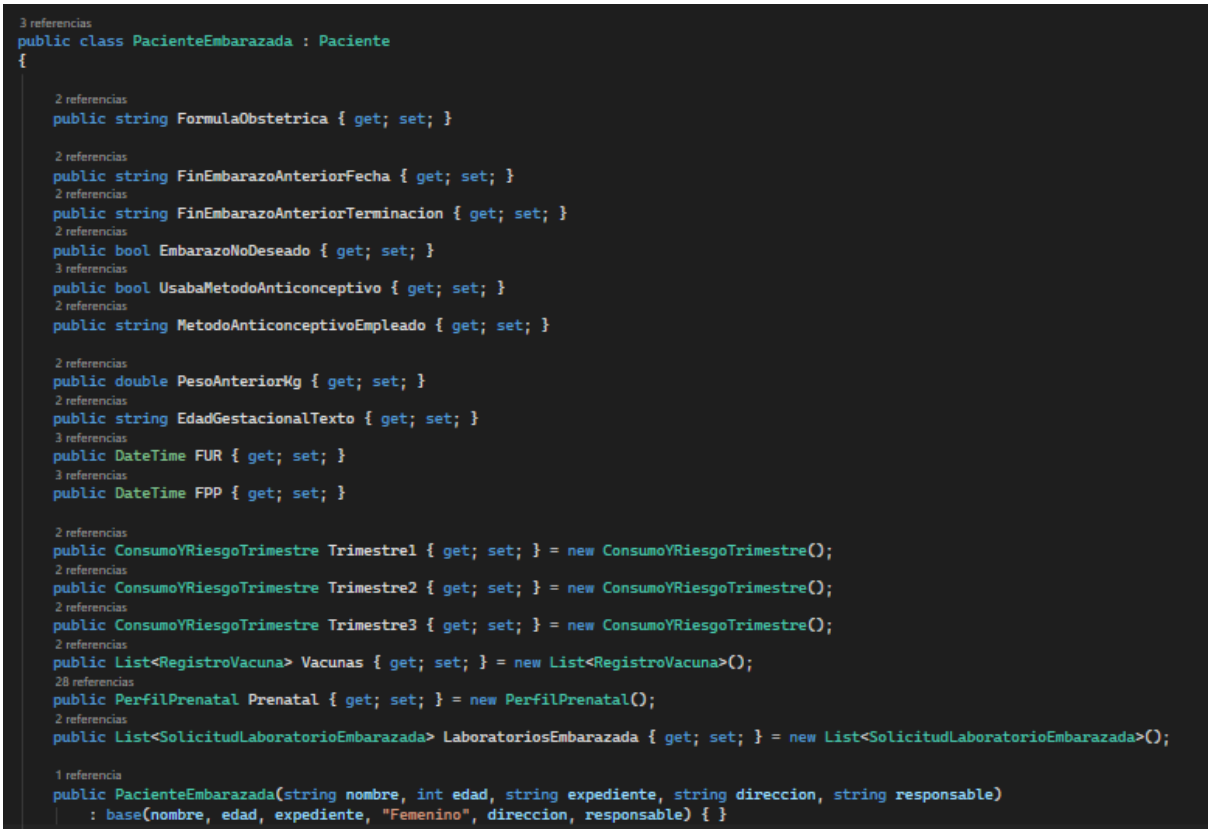
    // Datos generales
    4 referencias
    public string Nombre { get; set; }
    2 referencias
    public int Edad { get; set; }
    5 referencias
    public string NumeroExpediente { get; set; }
    2 referencias
    public string Sexo { get; set; }
    2 referencias
    public string Direccion { get; set; }
    2 referencias
    public string Responsable { get; set; }
    1 referencia
    public DateTime FechaConsulta { get; set; } = DateTime.Now;

    // Motivo de Consulta
    2 referencias
    public string ConsultaPor { get; set; }
    2 referencias
    public string PresenteEnfermedad { get; set; }

    // Signos Vitales y Medidas Antropométricas
    2 referencias
    public double TemperaturaCelsius { get; set; }
    2 referencias
    public int FrecuenciaRespiratoria { get; set; }
    2 referencias
    public int FrecuenciaCardiaca { get; set; }
    2 referencias
    public int Pulso { get; set; }
    2 referencias
    public string PresionArterial { get; set; }
    2 referencias
    public double SaturacionOxigeno { get; set; }
    3 referencias
    public double PesoKg { get; set; }
    4 referencias
    public double TallaCm { get; set; }
    2 referencias
    public double CircunferenciaAbdominalCm { get; set; }
```

- Sobrecarga de constructores (SubclasesPacientes.cs)

  Se utilizó la sobrecarga de constructores con el objetivo de evitar la redundancia en las subclases especializadas PacienteEmbarazada, PacienteMujerEdadFeril. Estas subclases no solicitan el parámetro de sexo/género al usuario, en su lugar, se llena directamente la cadena fija “Femenino” hacia el constructor de la clase base mediante la palabra clave: base(...)



```
3 referencias
public class PacienteEmbarazada : Paciente
{

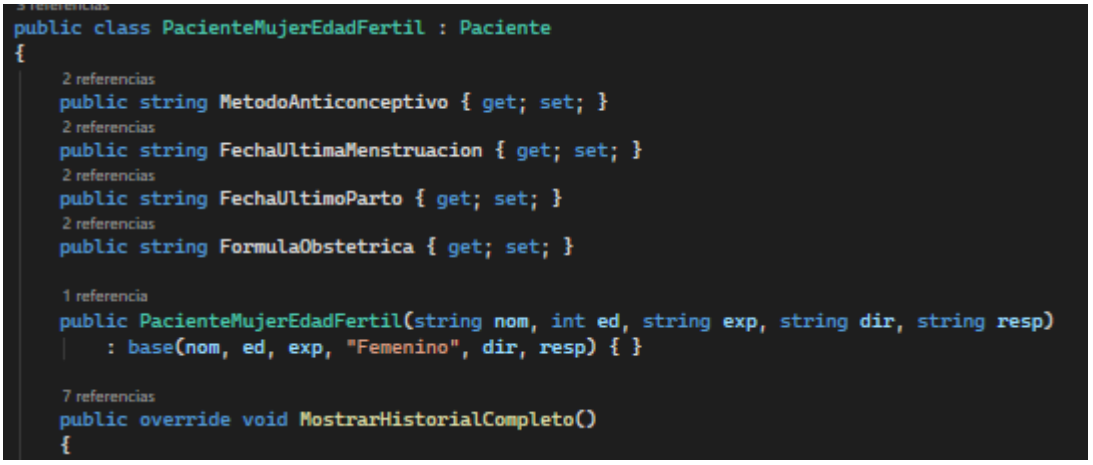
    2 referencias
    public string FormulaObstetrica { get; set; }

    2 referencias
    public string FinEmbarazoAnteriorFecha { get; set; }
    2 referencias
    public string FinEmbarazoAnteriorTerminacion { get; set; }
    2 referencias
    public bool EmbarazoNoDeseado { get; set; }
    3 referencias
    public bool UsabaMetodoAnticonceptivo { get; set; }
    2 referencias
    public string MetodoAnticonceptivoEmpleado { get; set; }

    2 referencias
    public double PesoAnteriorKg { get; set; }
    2 referencias
    public string EdadGestacionalTexto { get; set; }
    3 referencias
    public DateTime FUR { get; set; }
    3 referencias
    public DateTime FPP { get; set; }

    2 referencias
    public ConsumoYRiesgoTrimestre Trimestre1 { get; set; } = new ConsumoYRiesgoTrimestre();
    2 referencias
    public ConsumoYRiesgoTrimestre Trimestre2 { get; set; } = new ConsumoYRiesgoTrimestre();
    2 referencias
    public ConsumoYRiesgoTrimestre Trimestre3 { get; set; } = new ConsumoYRiesgoTrimestre();
    2 referencias
    public List<RegistroVacuna> Vacunas { get; set; } = new List<RegistroVacuna>();
    28 referencias
    public PerfilPrenatal Prenatal { get; set; } = new PerfilPrenatal();
    2 referencias
    public List<SolicitudLaboratorioEmbarazada> LaboratoriosEmbarazada { get; set; } = new List<SolicitudLaboratorioEmbarazada>();

    1 referencia
    public PacienteEmbarazada(string nombre, int edad, string expediente, string direccion, string responsable)
        : base(nombre, edad, expediente, "Femenino", direccion, responsable) { }
```



```
public class PacienteMujerEdadFertil : Paciente
{
    2 referencias
    public string MetodoAnticonceptivo { get; set; }
    2 referencias
    public string FechaUltimaMenstruacion { get; set; }
    2 referencias
    public string FechaUltimoParto { get; set; }
    2 referencias
    public string FormulaObstetrica { get; set; }

    1 referencia
    public PacienteMujerEdadFertil(string nom, int ed, string exp, string dir, string resp)
        : base(nom, ed, exp, "Femenino", dir, resp) { }

    7 referencias
    public override void MostrarHistorialCompleto()
    {
```

- Destructores y liberación de memoria (Paciente.cs y Program.cs)

  Dado que C# gestiona automáticamente la memoria con Garbage collector, se implementó un destructor explícito llamado “~Paciente()”, con el objetivo de demostrar el ciclo de vida del objeto. Cuando los expedientes son eliminados de la lista en GestorExpediente.cs, se invocan los métodos de recolección en Program.cs para activar los finalizadores.

```
        // Destructor
        0 referencias
        ~Paciente()
        {
            Console.WriteLine($"\n[DESTRUCTOR]: El expediente N° {NumeroExpediente} de '{Nombre}' ha sido liberado de la memoria RAM.");
        }
    }
}
```

```
        0 referencias
        private static void DemostrarDestructor()
        {
            Console.WriteLine("\nLimpiando lista y forzando ejecución del Garbage Collector...");
            gestor.LimpiarLista();
            GC.Collect();
            GC.WaitForPendingFinalizers();
            Console.WriteLine("Memoria liberada exitosamente.");
        }
    }
}
```

4.2. Estructura de datos (GestorExpedientes.cs)

Uso de List<T> (List<Paciente>).

Para administrar los datos de memoria sin depender de arreglos estáticos rígidos, se implementó la colección genérica List<T> (List<Paciente>) dentro de la clase GestorExpediente. Esto nos permite la inserción dinámica en tiempo de ejecución sin definir un límite superior fijo, y a su misma vez, nos brinda soporte al polimorfismo definiendo (List<Paciente>), la estructura puede almacenar instancias heterogéneas (PacienteInfantil, PacienteAdolescente, PacienteEmbarazada). Y al iterar la lista, se ejecuta el método “MostrarHistorialCompleto()” correspondiente a la clase concreta instanciada.

```
namespace Prueba_Proyecto_3
{
    2 referencias
    public class GestorExpedientes
    {
        5 referencias
        private List<Paciente> listaPacientes = new List<Paciente>();
```

```
public class PacienteAdolescente : Paciente
{

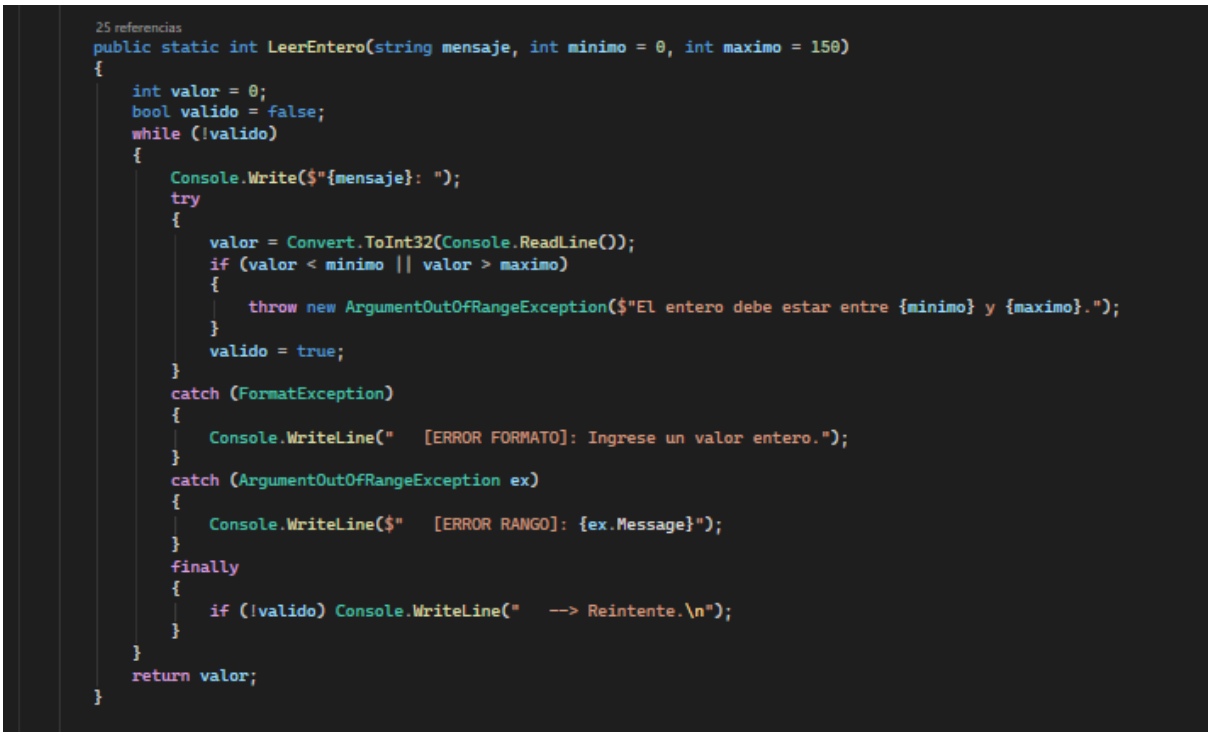
    2 referencias
    public Dictionary<string, bool> AntecedentesFamiliares { get; set; } = new Dictionary<string, bool>();


    3 referencias
    public List<AntecedentePersonal> AntecedentesPersonales { get; set; } = new List<AntecedentePersonal>();
```

4.3. Manejo de Excepciones (try-catch-finally).

Todo el flujo de lectura de datos e interacción con el usuario fue aislado en la clase auxiliar “CapturadorDatos.cs” . Esto garantiza que los errores de tipeo o desbordamiento no detengan abruptamente la ejecución del sistema,

implementando bloques try-catch-finally.



```
25 referencias
public static int LeerEntero(string mensaje, int minimo = 0, int maximo = 150)
{
    int valor = 0;
    bool valido = false;
    while (!valido)
    {
        Console.Write($"{mensaje}: ");
        try
        {
            valor = Convert.ToInt32(Console.ReadLine());
            if (valor < minimo || valor > maximo)
            {
                throw new ArgumentOutOfRangeException($"El entero debe estar entre {minimo} y {maximo}.");
            }
            valido = true;
        }
        catch (FormatException)
        {
            Console.WriteLine("   [ERROR FORMATO]: Ingrese un valor entero.");
        }
        catch (ArgumentOutOfRangeException ex)
        {
            Console.WriteLine($"   [ERROR RANGO]: {ex.Message}");
        }
        finally
        {
            if (!valido) Console.WriteLine("   --> Reintente.\n");
        }
    }
    return valor;
}
```

Algunos ejemplos de excepciones.

Excepciones Específicas Prevenidas.	Explicación
FormatException	Se activa si el usuario ingresa letras en campos númericos (Convert.ToInt32, Convert.ToDouble) o fechas con formato incorrecto (DateTime.ParseExact).
ArgumentOutOfRangeException	Se lanza manualmente mediante throw new ArgumentOutOfRangeException(). si los valores numéricos están fuera del rango clínico estándar (edades negativas o temperaturas corporales superiores a 43 grados celsius).
OverflowException	Captura números enteros que sobrepasen el rango permitido por el tipo int.
ArgumentException e InvalidOperationException	Controlan que los campos obligatorios de texto no se envíen vacíos y que las entradas de confirmación se restrinjan a opciones válidas (S/N).

5. Instructivo de Ejecución.

El propósito de este instructivo es detallar el procedimiento paso a paso para la correcta apertura, compilación y ejecución de la solución del proyecto en el entorno de desarrollo integrado de Microsoft Visual Studio.

Requisitos Previos e Instalación.

Para garantizar una correcta compilación y ejecución del sistema, se debe asegurar de cumplir con los siguientes requisitos mínimos del entorno:

- Entorno de Desarrollo (IDE): Microsoft Visual Studio 2019 o superior.
- Cargas de trabajo necesarias: Desarrollo de escritorio de .NET.
- Versión del Framework: .NET Framework 4.8 o superior.

Procedimiento para Abrir la Solución.

1. Descomprima el archivo Zip donde se encuentra la carpeta del proyecto.
2. Navegue a la carpeta principal del proyecto.
3. Localice el archivo con extensión .sln.
4. Haga doble clic sobre el archivo .sln o abra Visual Studio y seleccione la opción Archivo > Abrir > Proyecto o solución... seleccionando el archivo.
5. Finalmente confirme que todas las clases y dependencias se hayan cargado correctamente revisando el panel del Explorador de soluciones.

Compilación.

1. Diríjase al menú superior y seleccione Compilar.
2. Haga clic en Limpiar solución.
3. Seleccione Compilar solución o presione la combinación de teclas Ctrl + Mayús + B.
4. Verifique que en la pestaña Salida situada en la parte inferior que no existan errores de compilación.
5. Presione la tecla F5 o haga clic en el botón verde Iniciar en la barra de herramientas superior para ejecutar en modo depuración.